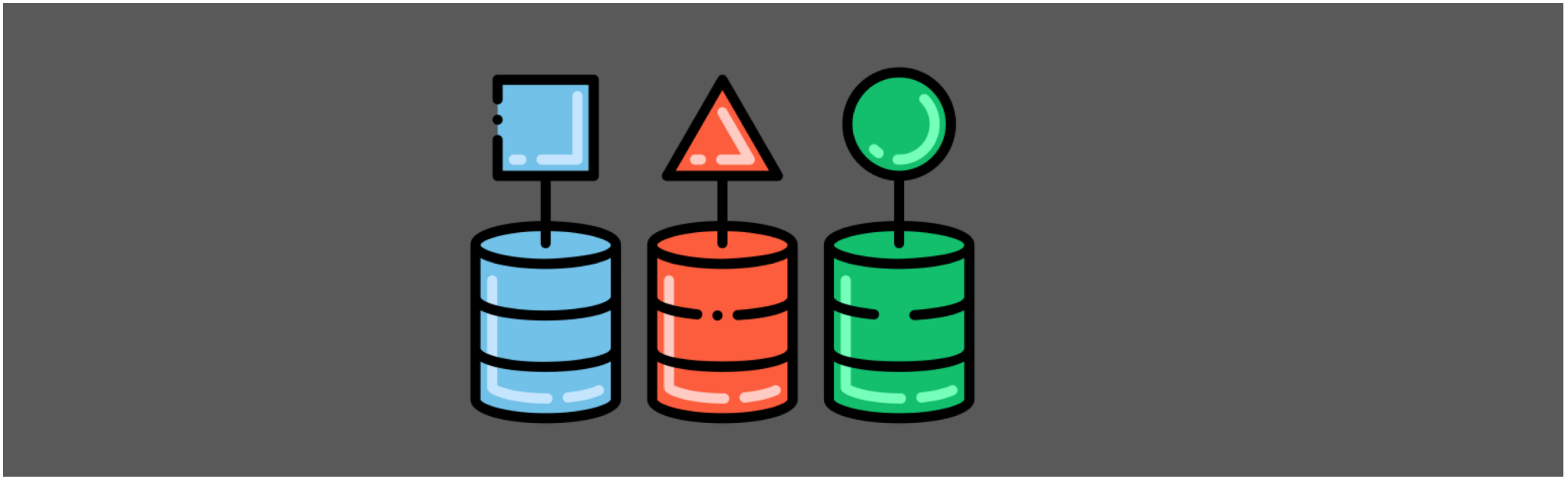


Section 3.1 – Redshift Objects

Agenda

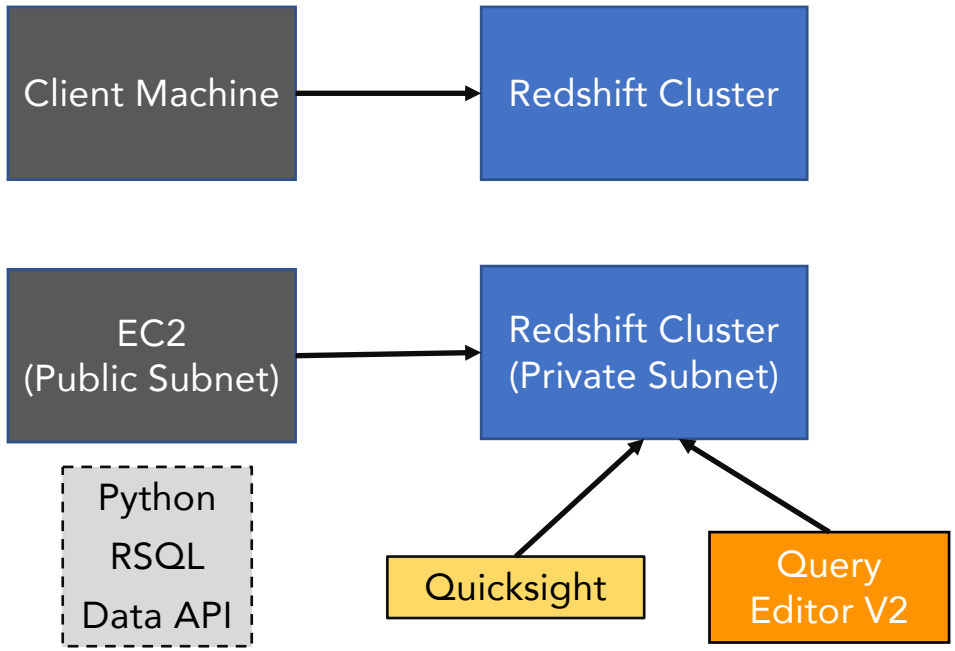
- Querying Database and Objects
- Data types
- DDL - Create/Alter/Drop/Show Objects
  - Database
  - Schema
  - Table
  - View
- SHOW command
- Load small sample data
- Transaction - BEGIN … COMMIT
- DML – INSERT, MERGE, PREPARE, SELECT, SELECT INTO, UPDATE, DELETE
- Cursors
- SQL Functions

Redshift and PostgreSQL

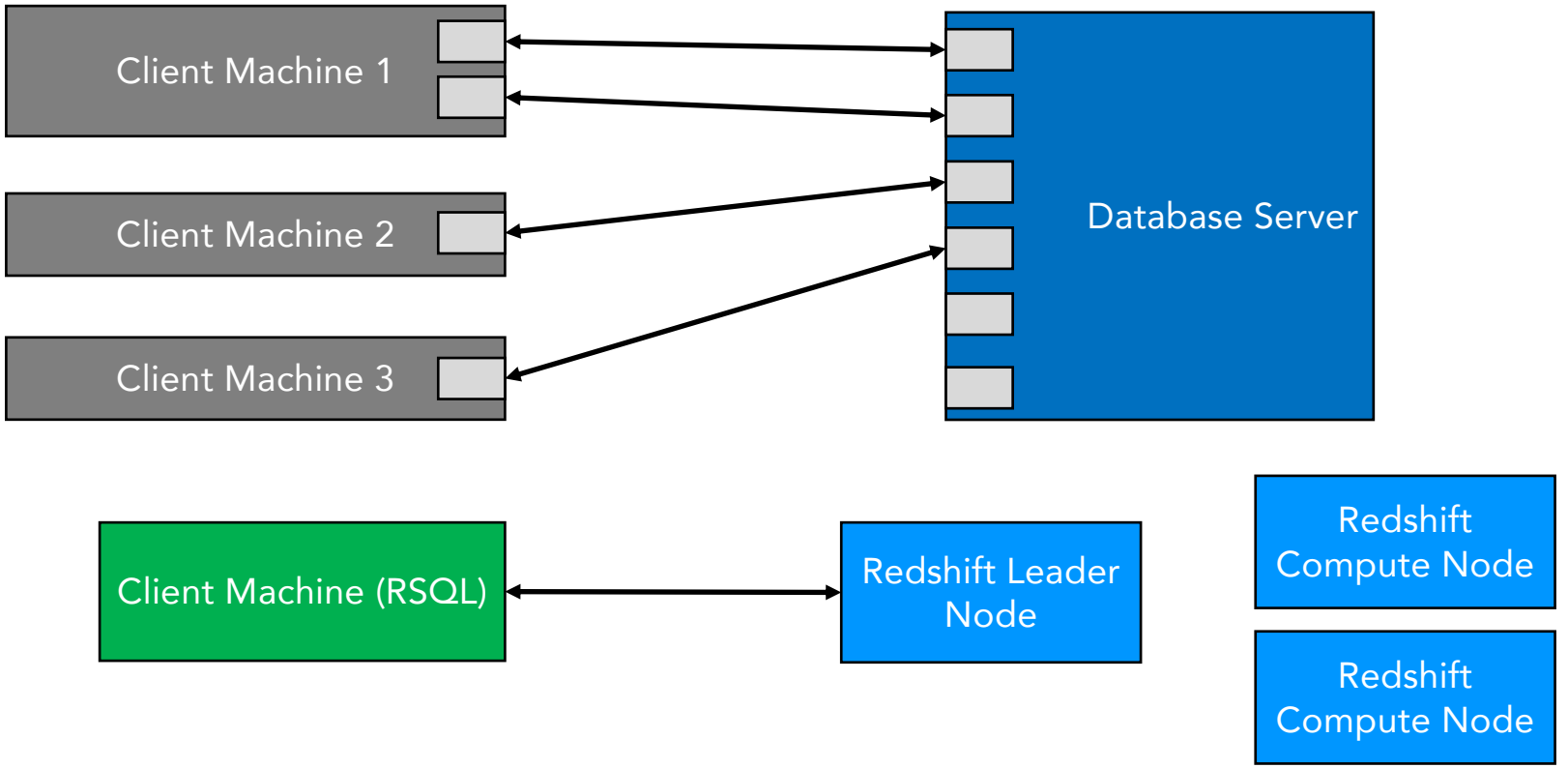
- Fork of PostgreSQL
- Redshift is meant for OLAP (DWH). PostgreSQL is for OLTP.
- Redshift is columnar storage. PostgreSQL is row oriented storage.
- PG features that are NOT supported
  - Indexes
  - Table partitioning
  - Triggers
  - Full text search
  - Some data types and fuctions

Querying Redshift

- JDBC
- Python
- ODBC
- RSQL
- Data API
- Third-party tools
- BI tools
  - Power BI
  - Tableau
  - Looker
  - Sisense
  - Quicksight
  - SRSS



Database Connection



RSQL

- Command Line Interface (CLI) for Redshift.
- Single Sign-On (SSO) authentication using ADFS, Okta, SAML/JWT based identity providers.
- You can describe properties or attributes of Amazon Redshift objects such as
  - Distribution key
  - Sort key
  - Materialized views
  - External tables in AWS Glue catalog or Hive Metastore
  - Redshift data sharing

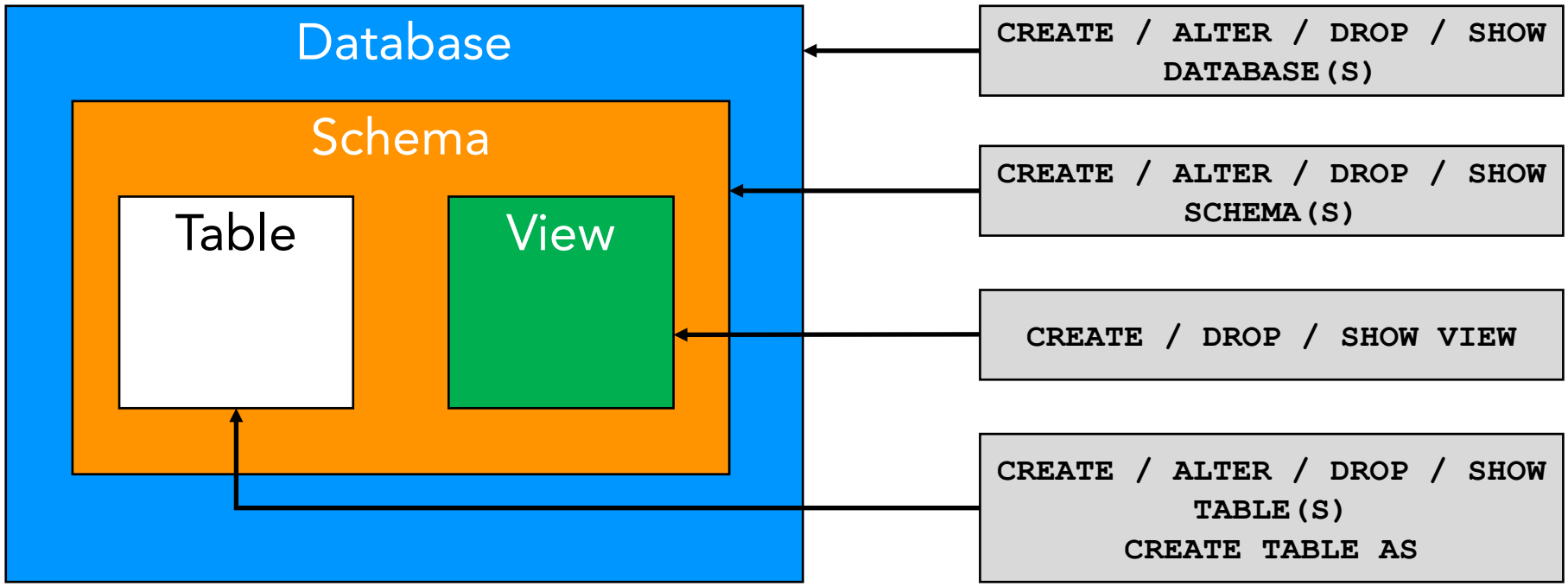
RSQL install

- Install OpenSSL
  - sudo yum install unixODBC openssl
- Install Redshift ODBC driver
- Set environment variables
- Install RSQL

Query Editor V2

- Available on the console

Object Hierarchy



Primary Key, Foreign Key, Unique Key

- A Primary key contains one or more columns that uniquely identifies each row in the table.
- A Foreign key is one or more columns in one table, that refers to the another table using Primary Key.
- A unique key ensures the values in one or more columns are unique across all rows in table.
- In Redshift all are informational and are not enforced.
- Important as these are used by query planner.

Redshift Data Types

Data Type	Storage (Bytes)	Range
SMALLINT, INT2	2	-32768 to +32767
INTEGER, INT, INT4	4	-2147483648 to +2147483647
BIGINT, INT8	8	-9223372036854775808 to 9223372036854775807
DECIMAL(p,s)	p	Based on precision and scale
REAL, FLOAT4	NA	1E-37 to 1E+37
DOUBLE, FLOAT8	NA	1E-307 to 1E+308

Data Type	Storage (Bytes)	Range
CHAR(n)	n	Based on “n”
VARCHAR(n)	1 - n	Based on what is stored
TEXT	256	Converted to VARCHAR(256)

Redshift Data Types

Data Type	Storage (Bytes)	Range
DATE	4	4713 BC to 294276 AD
TIME	8	00:00:00 to 24:00:00
TIMETZ	8	00:00:00+1459 to 00:00:00+1459 (Default UTC)
TIMESTAMP	8	4713 BC to 294276 AD
TIMESTAMPTZ	8	4713 BC to 294276 AD

Data Type	Storage (Bytes)	Range
BOOLEAN	1	True or False
HLLSKETCH (HyperLogLog)	1 - n	Converts to JSON type
VARBYTE (Variable length binary)	256	Can be casted to CHAR, VARCHAR, SMALLINT, INTEGER, BIGINT

Hands-on

Table Operations

- SELECT
- UPDATE
- INSERT
- DELETE
- JOINs
- CREATE TABLE AS (CTAS)

View

- Stores a complex SELECT statement.
- Create VIEW on top of it

```
SELECT a.c1, b.c1, ...
FROM t1 a, t2 b, t3 c, t4 d
WHERE a.c1 = b.c1
AND   b.c1 = c.c2
AND ...
```

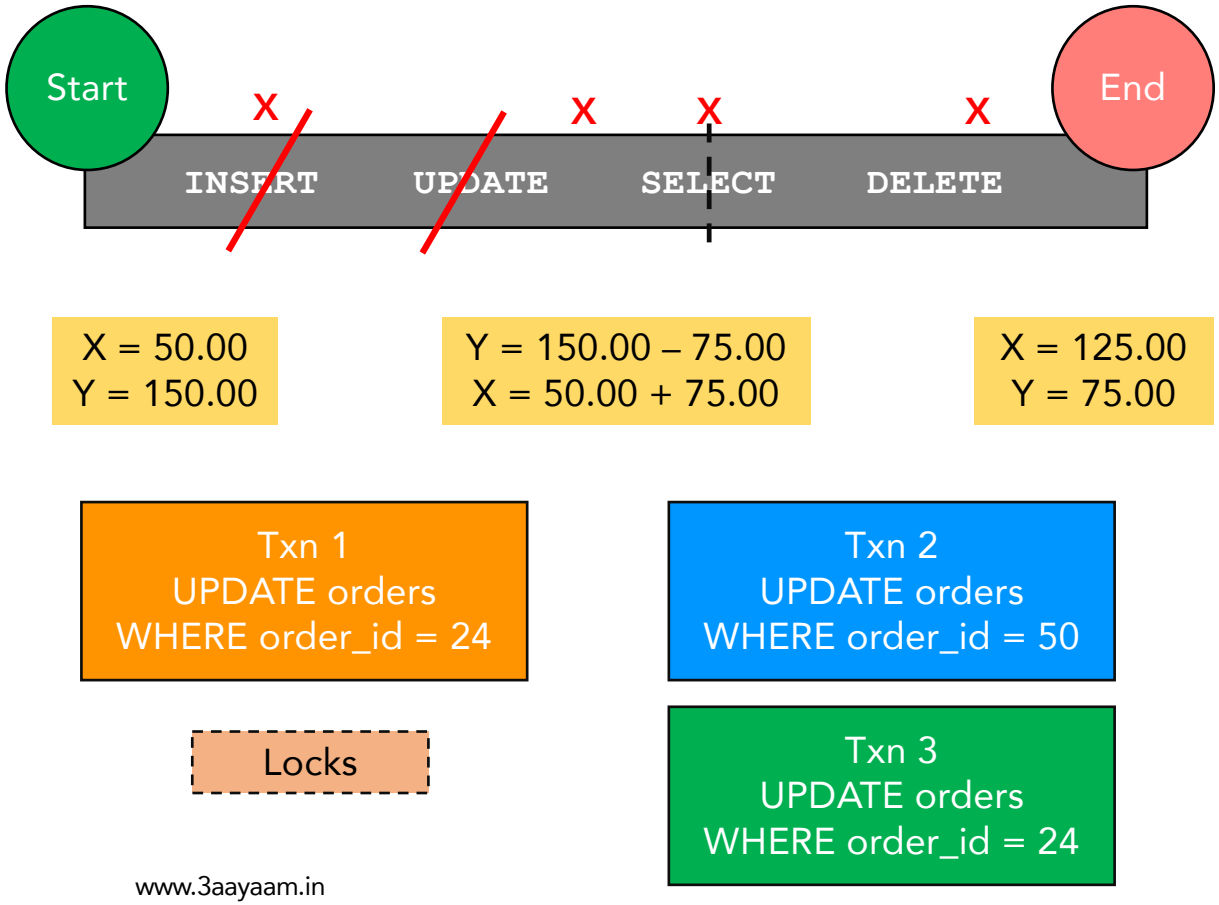
```
CREATE VIEW v1 AS
SELECT a.c1, b.c1, ...
FROM t1 a, t2 b, t3 c, t4 d
WHERE a.c1 = b.c1
AND   b.c1 = c.c2
AND ...;

SELECT * FROM v1;
```

Hands-on

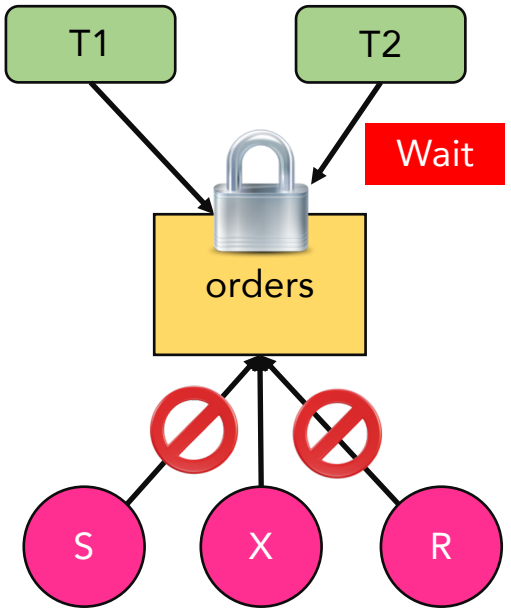
Transaction

- ACID
  - Atomicity – All happens or none of them happens. All or nothing.
  - Consistency – Data must be consistent before and after the transaction.
  - Isolation – Concurrent transactions do not interfere with each other.
  - Durability – All the changes are successful and persisted after a transaction completes. It will not be lost even if system failure occurs. STORAGE.



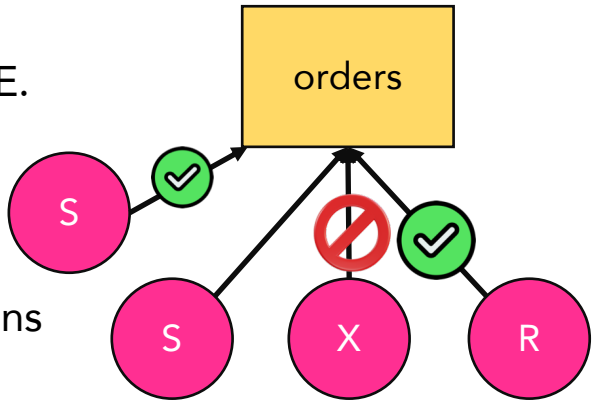
Locks and Isolation Levels

- Serializable isolation
  - Transactions are executed one after the other
- Locks
- AccessExclusiveLock (X)
  - DDL operations, such as ALTER TABLE, DROP, or TRUNCATE.
  - Blocks all other locking attempts.
- AccessShareLock (S)
  - UNLOAD, SELECT, UPDATE, or DELETE operations.
  - Blocks only AccessExclusiveLock. Doesn't block other sessions that are trying to read or write on the table.
- ShareRowExclusiveLock (R)
  - COPY, INSERT, UPDATE, or DELETE operations.
  - Blocks AccessExclusiveLock and other ShareRowExclusiveLock.



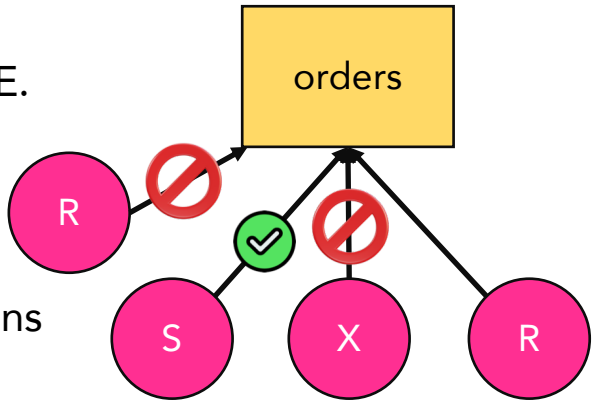
Locks and Isolation Levels

- Serializable isolation
  - Transactions are executed one after the other
- Locks
- AccessExclusiveLock (X)
  - DDL operations, such as ALTER TABLE, DROP, or TRUNCATE.
  - Blocks all other locking attempts.
- AccessShareLock (S)
  - UNLOAD, SELECT, UPDATE, or DELETE operations.
  - Blocks only AccessExclusiveLock. Doesn't block other sessions that are trying to read or write on the table.
- ShareRowExclusiveLock (R)
  - COPY, INSERT, UPDATE, or DELETE operations.
  - Blocks AccessExclusiveLock and other ShareRowExclusiveLock.

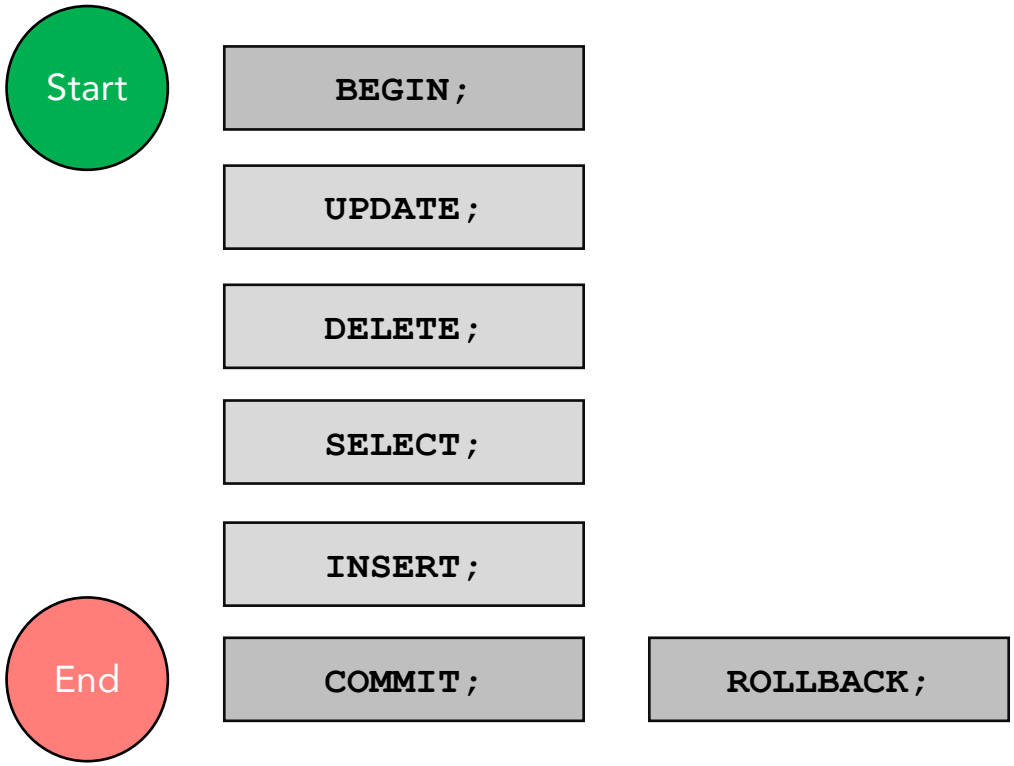


Locks and Isolation Levels

- Serializable isolation
  - Transactions are executed one after the other
- Locks
- AccessExclusiveLock (X)
  - DDL operations, such as ALTER TABLE, DROP, or TRUNCATE.
  - Blocks all other locking attempts.
- AccessShareLock (S)
  - UNLOAD, SELECT, UPDATE, or DELETE operations.
  - Blocks only AccessExclusiveLock. Doesn't block other sessions that are trying to read or write on the table.
- ShareRowExclusiveLock (R)
  - COPY, INSERT, UPDATE, or DELETE operations.
  - Blocks AccessExclusiveLock and other ShareRowExclusiveLock.



Transaction Execution

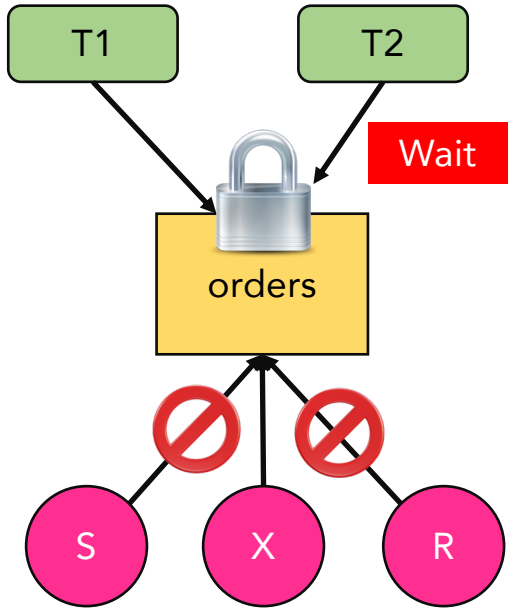


Check Transactions and Locks

- SVV_TRANSACTIONS
- PG_LOCKS

Transaction Hands-on

Scenario 1 & 2



Session 1

```
BEGIN;
UPDATE
DELETE
SELECT
ALTER
COMMIT;
```

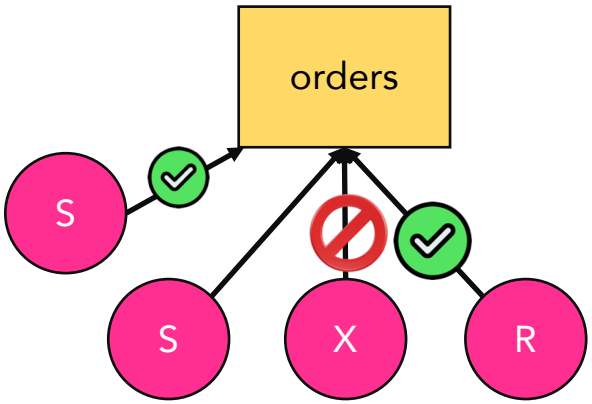
```
BEGIN;
ALTER TABLE
COMMIT;
```

Session 2

```
BEGIN;
UPDATE
DELETE
SELECT
ALTER
COMMIT;
```

```
BEGIN;
SELECT
DELETE
COMMIT;
```

Scenario 3



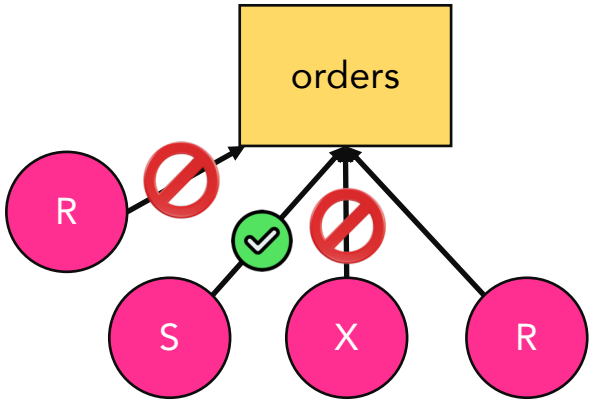
Session 1

```
BEGIN;
SELECT table
COMMIT;
```

Session 2

```
BEGIN;
DELETE
SELECT
ALTER TABLE
COMMIT;
```

Scenario 4



Session 1

```
BEGIN;
INSERT
COMMIT;
```

Session 2

```
BEGIN;
DELETE
SELECT
ALTER TABLE
COMMIT;
```

SUPER Data Type

- Semi-structured or document values. JSON.
- Can store up to 16 MB of data in a single column.
- Query SUPER data

```
emp_id
emp_name
date_of_joining
technology
{
    "cloud" : "AWS",
    "code" : ["python", "java"],
    "exp" : 10
}
```

```
SELECT emp_id,
       technology.cloud,
       technology.code[0],
       technology.exp
FROM   table
WHERE  emp_id = 123
```

```
123, AWS, python, 10
```

SUPER Hands-on

Source dataset that will be used to load the table

Cursors

- Area to hold the output of a SELECT statement.
- Retrieve a set of rows from the output. Provides a pointer.
- Result set is materialized on Leader Node.
- Cursor with large result set will have performance impact.

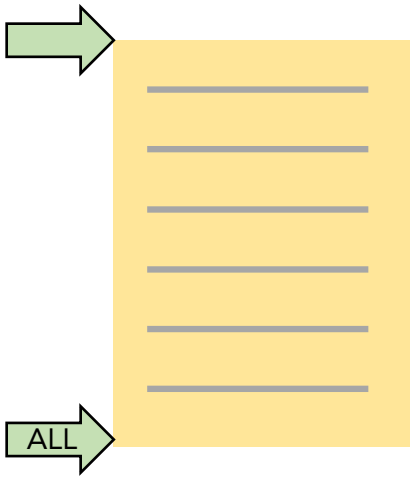
Node type	Maximum result set per cluster (MB)
RA3 16XL multiple nodes	14400000
DC2 Large single node	8000
DC2 Large multiple nodes	192000
DC2 8XL multiple nodes	3200000
RA3 4XL multiple nodes	3200000
RA3 XLPLUS multiple nodes	1000000
RA3 XLPLUS single node	64000
Amazon Redshift Serverless	150000

Cursors

```
DECLARE cursor CURSOR FOR query
```

```
FETCH NEXT, ALL, FORWARD FROM
cursor
```

```
CLOSE cursor
```

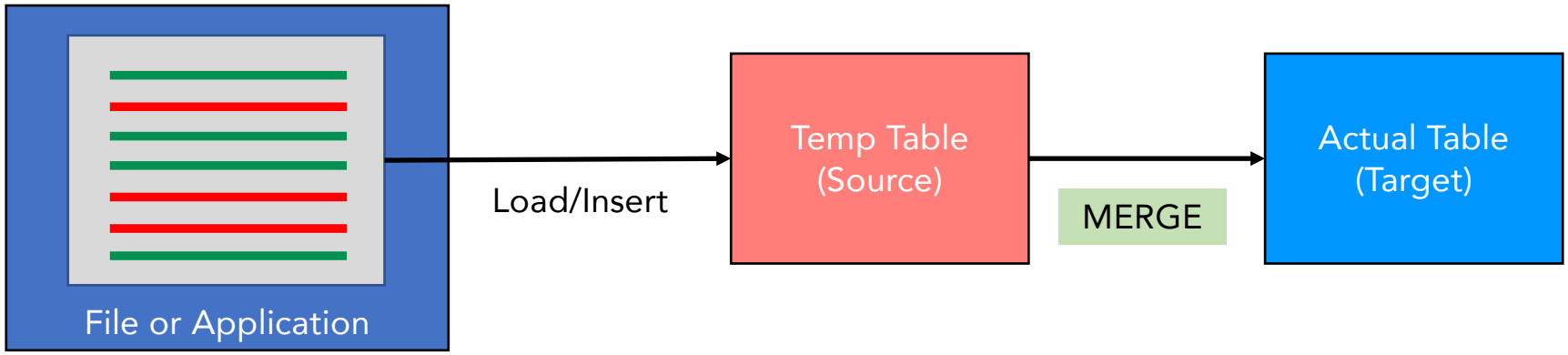


- STV_CURSOR_CONFIGURATION
- STV_ACTIVE_CURSORS

Cursor Hands-on

MERGE

- Upsert
  - UPDATE row when available
  - INSERT if row not available
- From SOURCE table to a TARGET table



MERGE

```
MERGE INTO target_table
USING source_table
ON match_condition
WHEN MATCHED THEN UPDATE SET col_name =  expr | DELETE
WHEN NOT MATCHED THEN INSERT ( col_name,... ) VALUES (...)
REMOVE DUPLICATES
```

- Rows in target_table can't match multiple rows in source_table.

MERGE Hands-on

PREPARE → EXECUTE

- Prepares a query plan for SQL statement using PREPARE command
- Why??
- SQL statement – SELECT, UPDATE, INSERT or DELETE
- Executes the statement using EXECUTE command
- Can take parameters whose values are substituted during run time.

```
PREPARE plan_name (datatype, ... ) AS sql_statement
```

```
EXECUTE plan_name (parameter, ...)
```

PREPARE EXECUTE Hands-on

Built-in Redshift SQL Functions

- AVG, COUNT, MAX, MIN, SUM
- ARRAY, ARRAY_CONCAT, ARRAY_FLATTEN, SPLIT_TO_ARRAY
- CASE
- CAST, CONVERT, TO_CHAR, TO_DATE, TO_NUMBER
- CAN_JSON_PARSE, JSON_PARSE, JSON_SERIALIZE, IS_VALID_JSON
- SQRT, CEIL, SIN, COS, LN, LOG, RANDOM, ROUND, POWER
- BTRIM, LTRIM, RTRIM, TRIM, UPPER, LEN, CONCAT, POSITION, REVERSE, SPLIT_PART

DATE SQL Functions

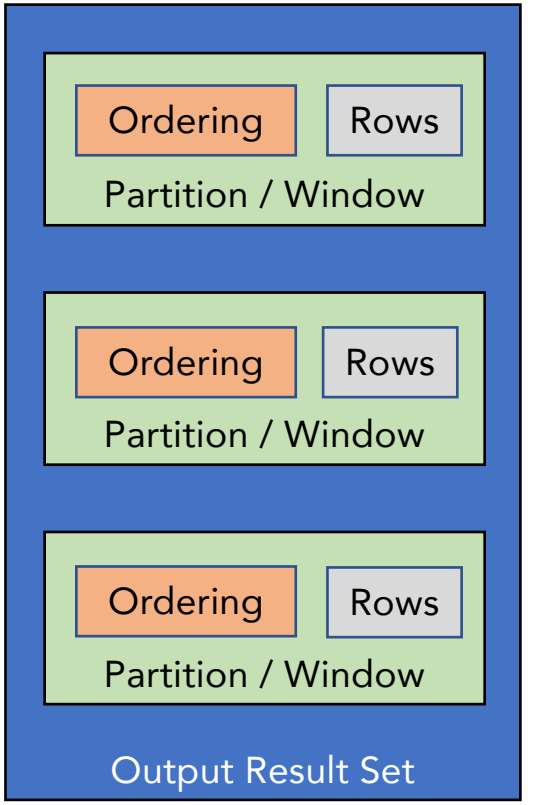
- CURRENT_DATE
- DATEADD
- EXTRACT
- ADD_MONTHS
- AT TIME ZONE
- CONVERT_TIMEZONE
- LAST_DAY
- TIMEOFDAY
- TO_DATE
- TO_TIMESTAMP
- TRUNC
- NEXT_DAY
- TIMESTAMP_CMP
- TIMESTAMP_CMP_DATE

Window Functions

- Partition or Window of result set
- PARTITION BY clause
- ORDER BY clause
- ROWS clause

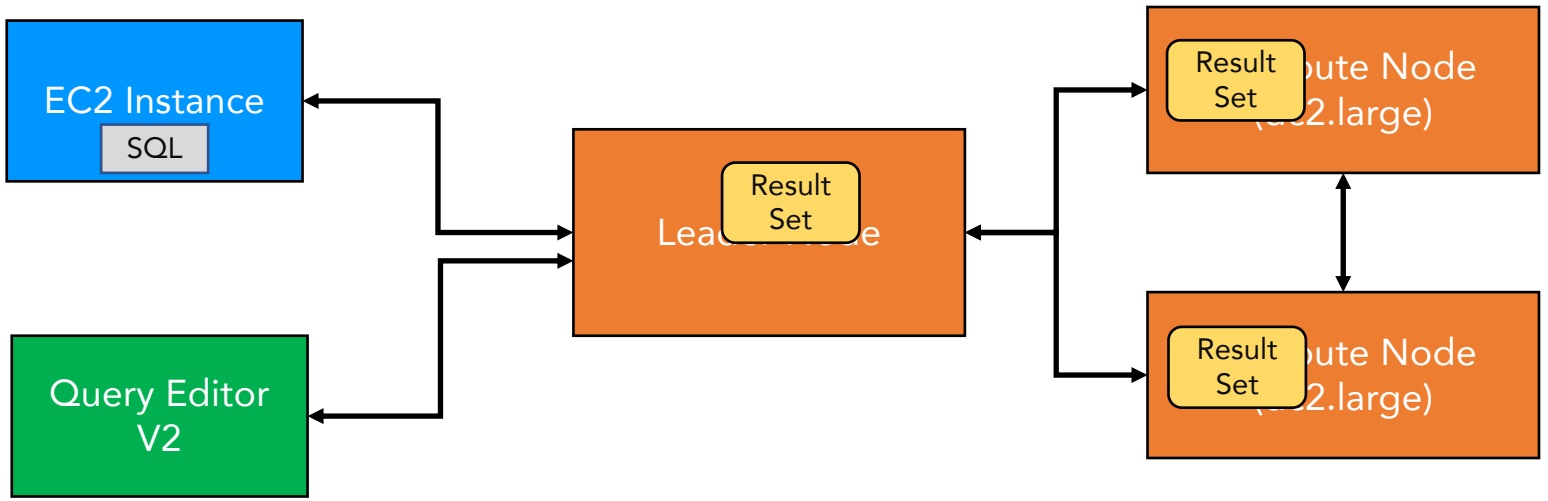
```
agg_func(col) OVER (PARTITION BY col1, col2 ORDER BY col3, col4)
```

- Agg_func
  - AVG, COUNT, MAX, MIN, SUM, DENSE_RANK, RANK, ROW_NUMBER, PERCENT_RANK



Window Function Hands-on

How queries are running in the cluster



Summary

- Query Editor V2 (from console) and RSQL (from client machine)
- Object hierarchy, Primary key, Foreign key and Unique key
- Data Types
- Table operations, Views
- Transactions, ACID, Locks/Isolation Levels
- SUPER data type
- Cursors
- Merge
- Prepare & Execute
- Built-in functions, Date functions, Window functions
- Command sheet